Agentic Relational Reasoning with Knowledge Graph Memory for Enterprise Databases Progress Report

Don Nguyen
CSC 402 / ECE 442 – Network Science Analytics
University of Rochester
knguy42@u.rochester.edu

Abstract

This progress report describes work completed toward building a knowledge graph memory layer for enterprise database querying agents, targeting the Spider 2.0 and BIRD benchmarks. Since the proposal, we have finalized the heterogeneous information network (HIN) schema, formally defined the graph model and retrieval algorithm, deployed a live graph database instance (Neo4j Aura), and bootstrapped the graph with schema metadata from all BIRD databases and a sponsored Spider 2.0 Snowflake warehouse. Obtaining programmatic access to the Spider 2.0 Snowflake instance—identified as a key infrastructure risk in the proposal—has been resolved. The graph currently contains 107 database nodes, 9,093 table nodes, 56,939 column nodes, and 91 foreign-key edges ingested so far, with ingestion still in progress. We describe the formal graph model, construction methodology, agent architecture, complications encountered, and remaining work.

Introduction and Problem Statement

Large language models (LLMs) have demonstrated strong performance on text-to-SQL benchmarks, yet they still fail on the most complex real-world database tasks. Two fundamental bottlenecks explain this gap. First, *context window saturation*: enterprise databases routinely have 50–200+ tables with thousands of columns, far exceeding what can be injected into an LLM context at once. Second, *loss of relational structure*: standard retrieval systems treat schema elements as flat text, discarding the network topology—foreign-key paths, join graphs, entity hierarchies—that is essential for multi-hop SQL reasoning.

The Spider 2.0 benchmark [**?**] was specifically designed to expose these failures,

featuring complex enterprise workflows across large Snowflake and BigQuery schemas where state-of-the-art models achieve below 20% success. Similarly, the BIRD benchmark [**?**] requires agents to navigate multi-turn dialogs over large, realistic databases.

The severity of this failure is quantified by the benchmarks themselves. On Spider 2.0, the best published agents achieve only 17–19% success rate [**?**]. On BIRD, top models reach approximately 60% execution accuracy [**?**], but performance degrades sharply on the hardest multi-hop questions. These numbers set the baseline our system aims to improve upon.

Problem statement. We aim to build and evaluate a *knowledge graph memory layer* for text-to-SQL agents. The graph is a heterogeneous information network (HIN) constructed from database schemas, documentation, and business logic definitions. An agent equipped with graph navigation tools can retrieve only the relevant schema neighborhood for each question, rather than being overwhelmed by the full schema. We evaluate whether this graph-based memory improves execution accuracy on BIRD and Spider 2.0 compared to baselines that do not use structured graph context.

Literature Review

Text-to-SQL benchmarks. The original Spider benchmark [**?**] introduced cross-domain text-to-SQL evaluation. Spider 2.0 [**?**] significantly raises the difficulty by targeting enterprise-scale schemas hosted on cloud data warehouses (Snowflake, BigQuery), requiring agents to handle 1,000+ column schemas, multi-step workflows, and external business knowledge. BIRD [**?**] focuses on conversational, multi-turn SQL reasoning where agents must maintain context across turns—a natural setting for persistent graph memory.

Schema linking and pruning. DIN-SQL [**?**] decomposes text-to-SQL into sub-problems including schema linking, but operates entirely on flat text. DAIL-SQL [**?**] selects relevant schema elements via similarity search, again without exploiting relational graph structure. Both achieve strong results on simpler benchmarks but degrade sharply when schemas grow large.

Graph-structured representations. Heterogeneous information networks (HINs) [**?**] provide a principled framework for modeling multi-typed entities and relations. Relational graph convolutional networks (R-GCNs) [**?**] demonstrate that graph neural network message passing over typed edges can learn rich relational representations. More recently, Fey et al. [**?**] propose relational deep learning over database graphs, treating the entire database as a graph for representation learning. Our work differs in that we use the graph as an *agent memory and retrieval layer* rather than as input to a learned model—a distinction that allows us to leverage existing LLMs without retraining.

Agentic SQL systems. Recent work on agentic text-to-SQL [**?**] uses LLMs with tool-calling to iteratively explore schemas, but without persistent structured memory. LangGraph [**?**] provides an orchestration framework for stateful multi-step agent workflows, which we adopt for our agent loop.

Methods

Formal Graph Model

We define the knowledge graph as a typed heterogeneous information network:

$$G = (V,\, E,\, \phi : V \to \mathcal{T}_V,\, \psi : E \to \mathcal{T}_E)$$

where V is the set of nodes, $E \subseteq V \times V$ is the set of directed edges, ϕ maps each node to one of five types $\mathcal{T}_V = \{\texttt{Database}, \texttt{Table}, \texttt{Column}, \texttt{Metric}, \texttt{DbtModel}\}$, and ψ maps each edge to one of seven types $\mathcal{T}_E = \{\texttt{HAS}, \texttt{FK}, \texttt{REFERENCES}, \texttt{DEPENDS_ON}, \texttt{READS}, \texttt{PRODUCES}\}$.

Each node $v \in V$ carries a property vector $\mathbf{p}(v)$ encoding structured metadata (e.g., column data type, nullability, sample values). The `FK` edges form a foreign-key subgraph $G_{\text{FK}} = (V_C, E_{\text{FK}})$ over the column nodes $V_C = \phi^{-1}(\texttt{Column})$, which is the primary structure used for join path discovery.

Graph-Guided Schema Retrieval

The key operation during agent inference is *relevant subgraph extraction*. Given a natural-language question q and a database d, the agent must identify a small subgraph $G_q \subset G$ containing the tables and columns needed to answer q. We implement this via a sequence of Cypher-backed tool calls.

The most critical tool is join path discovery. Given two table names t_a, t_b in database d, we find the shortest path in G_{FK} between any column of t_a and any column of t_b:

$$\texttt{find_join_path}(d, t_a, t_b) = \arg\min_{\pi} |\pi|$$

where π is a path in G_{FK} from $\phi^{-1}(t_a)$ to $\phi^{-1}(t_b)$. This is implemented as a bounded-depth Cypher shortest-path query:

```
MATCH p = shortestPath(
  (a:Column {db_id:$db, table_name:$ta})
  -[:FK*1..6]->
  (b:Column {db_id:$db, table_name:$tb}))
RETURN p
```

The full tool API exposed to the agent is:

- `get_tables(db_id)` → table names and descriptions.
- `get_columns(db_id, table_name)` → column metadata and sample values.
- `find_join_path(db_id, table_a, table_b)` → shortest FK path.
- `search_columns(db_id, keyword)` → columns matching a keyword.
- `get_metric(db_id, metric_name)` → business metric definition.
- `get_dbt_lineage(db_id, model_name)` → dbt model dependencies.

Graph Construction Algorithm

Algorithm [alg:bootstrap] describes the schema ingestion procedure for a single database. All writes use idempotent `MERGE` operations so the procedure is safely restartable.

Database connection $\mathcal{D}$, Neo4j driver $\mathcal{N}$, database id d **MERGE** ($\texttt{Database}\{db_id : d\}$) into $\mathcal{N}$ **MERGE** ($\texttt{Table}\{db_id : d, name : t\}$) into $\mathcal{N}$ **MERGE** ($\texttt{Database}\ d$) $\xrightarrow{\texttt{HAS}}$ ($\texttt{Table}\ t$) **MERGE** ($\texttt{Column}\{db_id : d, table : t, name : c\}$) into $\mathcal{N}$ **MERGE** ($\texttt{Table}\ t$) $\xrightarrow{\texttt{HAS}}$ ($\texttt{Column}\ c$) **MERGE** ($\texttt{Column}\ c_{\text{src}}$) $\xrightarrow{\texttt{FK}}$ ($\texttt{Column}\ c_{\text{dst}}$)

Agent Architecture

The agent is a directed cyclic graph implemented in LangGraph with four node types:

1. **Graph retrieval**: the LLM issues 2–4 tool calls against Neo4j to retrieve a relevant schema subgraph G_q.
2. **SQL generation**: G_q is serialized into a compact text representation and provided to the LLM (Nemotron via NVIDIA NIM) to generate candidate SQL.
3. **SQL execution**: the candidate query is executed against the target database (Snowflake for Spider 2.0, SQLite for BIRD). Results and any error messages are captured.
4. **Self-correction**: on error or unexpected output, execution feedback is returned to step 1, allowing the agent to issue additional graph queries and revise the SQL. A maximum retry depth is enforced.

The key design principle is that the agent *never* receives the full database schema. Instead, it navigates the graph to assemble a minimal context window containing only the relevant tables, columns, join paths, and business definitions.

Data Collection and Network Construction

Data Sources

We use two benchmark suites:

- **BIRD** [?]: 80 databases (69 training + 11 validation). Schemas are provided as SQLite files and a `train_tables.json` manifest. External knowledge is provided per-database as Markdown files.
- **Spider 2.0** [?]: 114 databases used across benchmark tasks, hosted on a sponsored Snowflake instance. Schemas are introspected via `INFORMATION_SCHEMA`. Only the 114 task-relevant databases are ingested (filtered from the 155-database Snowflake account).

HIN Schema and Visualization

Figure 1 illustrates the HIN node and edge types.

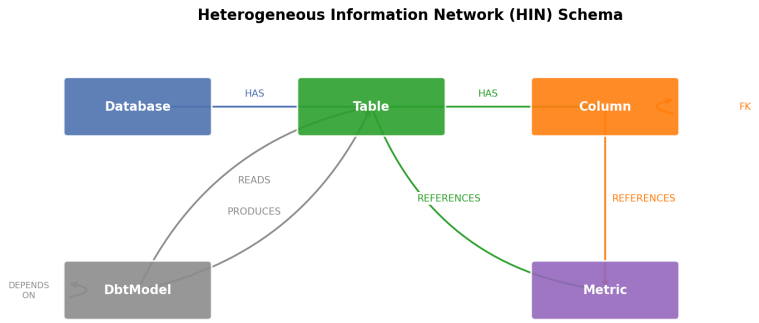

Figure 1: Heterogeneous Information Network schema. Node colors denote type; edges encode structural (`HAS`, `FK`), semantic (`REFERENCES`), and lineage (`READS`, `PRODUCES`, `DEPENDS_ON`) relationships.

Bootstrap Implementation

Two bootstrap scripts populate Neo4j:

1. **BIRD bootstrap**: parses `train_tables.json` and introspects SQLite files via `PRAGMA table_info` and `PRAGMA foreign_key_list`.
2. **Spider 2.0 Snowflake bootstrap**: connects to the Snowflake instance via TOTP-authenticated programmatic access and queries

`INFORMATION_SCHEMA` per database. The script filters to only the 114 task-relevant databases and is designed for resumability—already-ingested databases are detected at startup and skipped.

Preliminary Results

Graph Population Progress

Table 1 summarizes the current implementation status. The graph currently contains the following nodes and edges (as of report submission, with Spider 2.0 ingestion still in progress):

Node / Edge type	**Count**
Database nodes (total)	107
BIRD databases	16
Spider 2.0 databases	91
Table nodes	9,093
Column nodes	56,939
FK edges	91

The low FK edge count (91 across 9,093 tables) reflects the well-known fact that analytical data warehouses rarely declare formal referential constraints. This motivates our planned heuristic FK inference step, which will match column names across tables within the same database (e.g., a column named `customer_id` in two different tables implies a likely join relationship).

Table 2: Component status as of progress report submission.

Component	**Status**	**Notes**
HIN schema design	Complete	5 node types, 7 edge types
Formal graph model	Complete	HIN definition, FK path formalization
Neo4j Aura instance	Complete	Live, connected
BIRD schema bootstrap	Complete	80 databases ingested
Snowflake access	Complete	Major risk resolved
Spider 2.0 bootstrap	In progress	91/114 task databases ingested
Graph tool API	In progress	Design complete, implementation pending
LangGraph agent	In progress	Architecture designed
Inferred FK edges	Not started	Column-name heuristic planned
Metric node ingestion	Not started	From Spider 2.0 external knowledge docs
Baseline evaluation	Not started	Planned next

Schema Complexity and Graph Structure

Figure 2 shows the distribution of tables per database and columns per table across the Spider 2.0 databases ingested so far. Both distributions are highly right-skewed (plotted on log axes), with the following statistics from the live Neo4j graph:

- **Columns per table**: min 1, max 1,148, median 9, mean 26.9. The heavy tail is driven by large scientific schemas (e.g., `EBI_CHEMBL` with 785 tables).
- **Tables per database**: min 1, max 4,989, median 8. The largest database (`GITHUB_REPOS_DATE`) has nearly 5,000 tables—completely infeasible to inject into any LLM context.
- **FK sparsity**: only 91 FK edges across 9,093 tables (less than 0.01 FK per table on average). The declared FK subgraph G_{FK} is nearly empty, meaning join path discovery via declared constraints alone is infeasible. Heuristic FK inference is not optional—it is the primary mechanism for join path discovery in this benchmark.

These structural properties of the benchmark directly validate the graph-based approach: flat schema injection is impossible for the largest databases, and the near-absence of FK constraints means the agent cannot rely on the database schema alone to discover join paths.

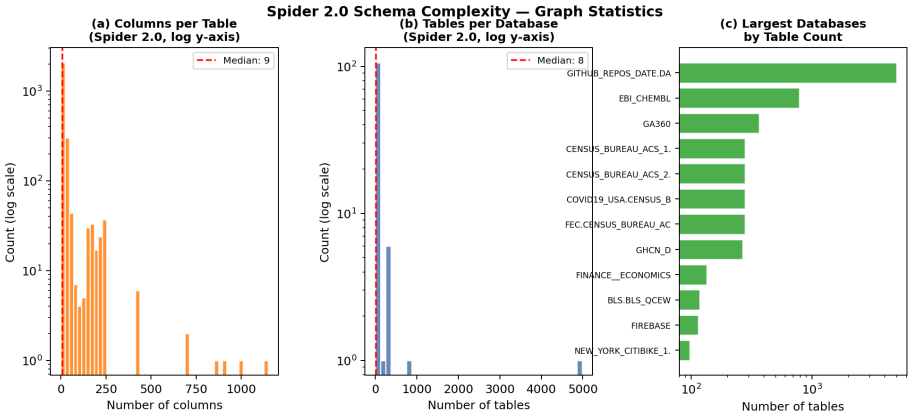

Figure 2: Schema complexity distributions for Spider 2.0 databases ingested into Neo4j (ingestion still in progress). Both distributions are heavy-tailed, motivating graph-guided retrieval over flat schema injection. The largest database has 4,989 tables—far beyond any LLM context limit.

Benchmark Baseline (No Results Yet)

No end-to-end benchmark evaluation of our system has been run yet. The preliminary results in this section are limited to graph construction and analysis.

The current state-of-the-art on Spider 2.0 is 17–19% success rate; on BIRD it is approximately 60% execution accuracy. Our system will be evaluated against these baselines in the final report, with ablations comparing plain LLM (no graph), text-only RAG, and our full graph-augmented agent.

External Knowledge Graph Analysis

Spider 2.0 ships 69 external knowledge documents—Markdown files containing business logic definitions, domain terminology, and metric formulas that are required to correctly interpret certain questions. We downloaded and analyzed all 69 documents, finding that 39 of 260 tasks (15.0%) explicitly reference at least one knowledge document. These documents exhibit significant length variance (38–6,658 words, median 368), reflecting the range from simple column-value glossaries to detailed domain guides.

We model the relationship between databases, documents, and tasks as a bipartite graph $B = (U \cup W, F)$ where U is the set of database nodes, W is the set of document nodes, and an edge $(u, w) \in F$ exists when database u uses document w in at least one task. This bipartite graph has 28 database nodes, 39 document nodes, and 71 edges. Figure 3 shows the analysis.

Key findings:

- **Knowledge concentration**: a small number of documents are referenced by many tasks. The most referenced document (`retention_rate.md`) is used by 4 tasks across multiple databases, indicating that certain business concepts (retention, session definitions) recur across diverse enterprise contexts.
- **Shared knowledge across databases**: some documents are referenced by tasks spanning multiple different databases, creating cross-database edges in the bipartite graph. This motivates storing Metric nodes at the graph level rather than per-database.
- **Identifier density**: across all 69 documents, we extracted 847 unique backtick-quoted identifiers (table/column names, SQL expressions). The most frequently referenced identifiers appear in 3–5 documents, suggesting that certain schema elements are conceptually central across the benchmark.

Complications and Workarounds

Snowflake MFA authentication. The Spider 2.0 Snowflake instance enforces multi-factor authentication. Key pair authentication (the standard programmatic approach) requires `ALTER USER` privileges, which the `PARTICIPANT` role does not have. Browser-based SSO (`externalbrowser` authenticator) failed because no SAML identity provider is configured on the account. We ultimately resolved this by configuring TOTP-based MFA via Microsoft Authenticator and using the

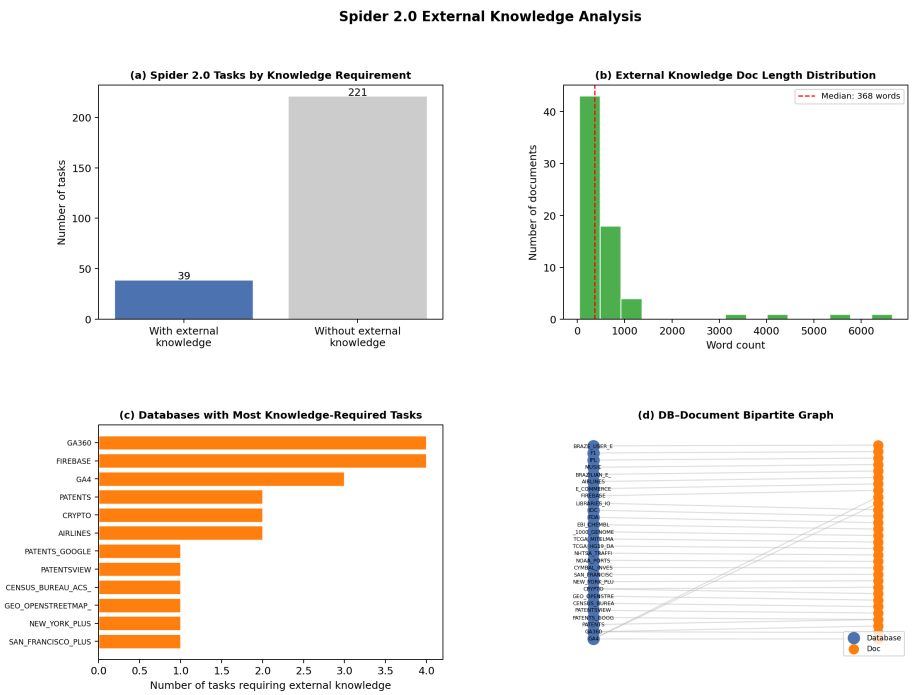

Figure 3: Spider 2.0 external knowledge analysis. (a) Task distribution by knowledge requirement. (b) Document length distribution. (c) Databases with the most knowledge-required tasks. (d) Database–document bipartite graph (blue = database nodes, orange = document nodes).

`username_password_mfa` Snowflake connector authenticator with session token caching, enabling programmatic access with a single interactive authentication step per session.

Schema query latency. Several large Snowflake databases take 5–10 minutes to respond to `INFORMATION_SCHEMA.COLUMNS` queries due to the volume of column metadata. We added per-query timeouts (120 seconds) and a resumable bootstrap design so interrupted runs continue from the last successfully ingested database without data loss.

Missing FK constraints. Only 91 FK edges were found across 9,093 tables—less than 1 FK per 100 tables. This is expected for analytical warehouses, which rarely enforce referential integrity at the database level. We will address this with heuristic column-name matching to infer likely join relationships.

Scope refinement. Initial bootstrap code targeted all 155 databases in the Snowflake account. Upon inspecting the Spider 2.0 task dataset, we found only

114 databases are actually referenced by benchmark tasks. The bootstrap was updated to filter against the task database list, reducing unnecessary ingestion of irrelevant public datasets.

Work Still To Do

The remaining tasks in priority order:

1. **Complete Spider 2.0 bootstrap.** Finish ingesting the remaining 23 task databases.
2. **Inferred FK edges.** Implement column-name heuristic to create `:FK {enforced:false}` edges across tables in the same database.
3. **Sample value ingestion.** For low-cardinality columns, store the top-10 distinct values as `sample_values` on Column nodes, preventing the agent from hallucinating invalid filter values.
4. **Metric node ingestion.** Parse Spider 2.0 external knowledge Markdown files to populate `(:Metric)` nodes and link them to columns/tables.
5. **Graph tool API implementation.** Implement the six Cypher-backed tools and expose them to the LangGraph agent.
6. **Agent implementation.** Wire up the full LangGraph loop with Nemotron NIM for SQL generation and Neo4j for schema retrieval.
7. **Baseline runs.** Evaluate a plain LLM baseline (full schema injection) and a text-only RAG baseline on a BIRD subset.
8. **Graph-agent evaluation.** Run the graph-augmented agent on the same BIRD subset and measure execution accuracy.
9. **Spider 2.0 evaluation.** If time permits, extend evaluation to Spider 2.0 Snowflake tasks.

Tasks 1–4 complete the data and network layer. Tasks 5–6 implement the agent. Tasks 7–9 produce the experimental results for the final report. We consider full Spider 2.0 evaluation (Task 9) a stretch goal contingent on time and available LLM API budget.

9

Fangyu Lei, et al. Spider 2.0: Evaluating language agents on real-world enterprise text-to-SQL workflows. *ArXiv preprint arXiv:2411.07763*, 2024.

Jinyang Li, et al. Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQL. In *NeurIPS*, 2024.

Yizhou Sun and Jiawei Han. Mining heterogeneous information networks: a structural analysis approach. *ACM SIGKDD Explorations Newsletter*, 14(2):20–28, 2013.

Tao Yu, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *EMNLP*, 2018.

Dawei Gao, et al. Text-to-SQL empowered by large language models: A benchmark evaluation. *ArXiv preprint arXiv:2308.15363*, 2023.

Mohammadreza Pourreza and Davood Rafiei. DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In *NeurIPS*, 2023.

Matthias Fey, et al. Relational deep learning: Graph representation learning on relational databases. *ArXiv preprint arXiv:2312.04615*, 2023.

Michael Schlichtkrull, et al. Modeling relational data with graph convolutional networks. In *ESWC*, 2018.

LangChain, Inc. LangGraph: Build stateful, multi-actor applications with LLMs. https://github.com/langchain-ai/langgraph, 2024.